

T6 - Seguimiento, control y métricas del proyecto

¿Cómo sabemos si lo que planificamos se está cumpliendo?

Proyecto Software

Grado en Ingeniería Informática

Escuela Politécnica de Teruel

Universidad de Zaragoza

Seguimiento vs. Control

- ¿Qué es para vosotros/as (ahora mismo) hacer el seguimiento del proyecto?
- ¿Qué es para vosotros/as (ahora mismo) hacer el control del proyecto?

Seguimiento

- Recoger datos reales del proyecto.
- Comparar con lo planificado.

Control

- Tomar decisiones cuando hay desviación.
- Ajustar alcance, tiempo o recursos.

Medir sin actuar no es gestionar. Actuar sin medir es improvisar.

¿Controlar qué exactamente?

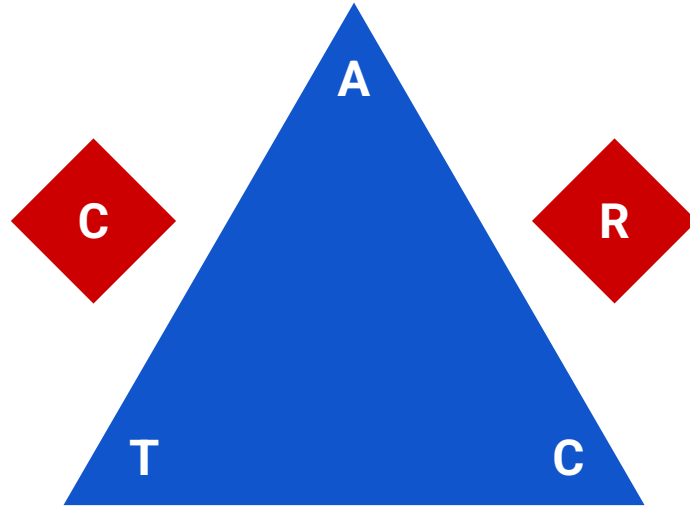
¿Qué dimensiones controlamos?

- **Alcance:** ¿Se ha ampliado sin decisión explícita?
- **Tiempo:** ¿Se están cumpliendo hitos?
- **Esfuerzo:** ¿Las tareas tardan lo previsto?
- **Riesgos:** ¿Se están materializando?
- **Calidad:** ¿Se está acumulando deuda técnica?

Un proyecto puede ir bien en una dimensión y mal en otra.

¿Controlar qué exactamente?

El equilibrio



Un proyecto es estable cuando estas dimensiones están en equilibrio.

Un proyecto como sistema interdependiente

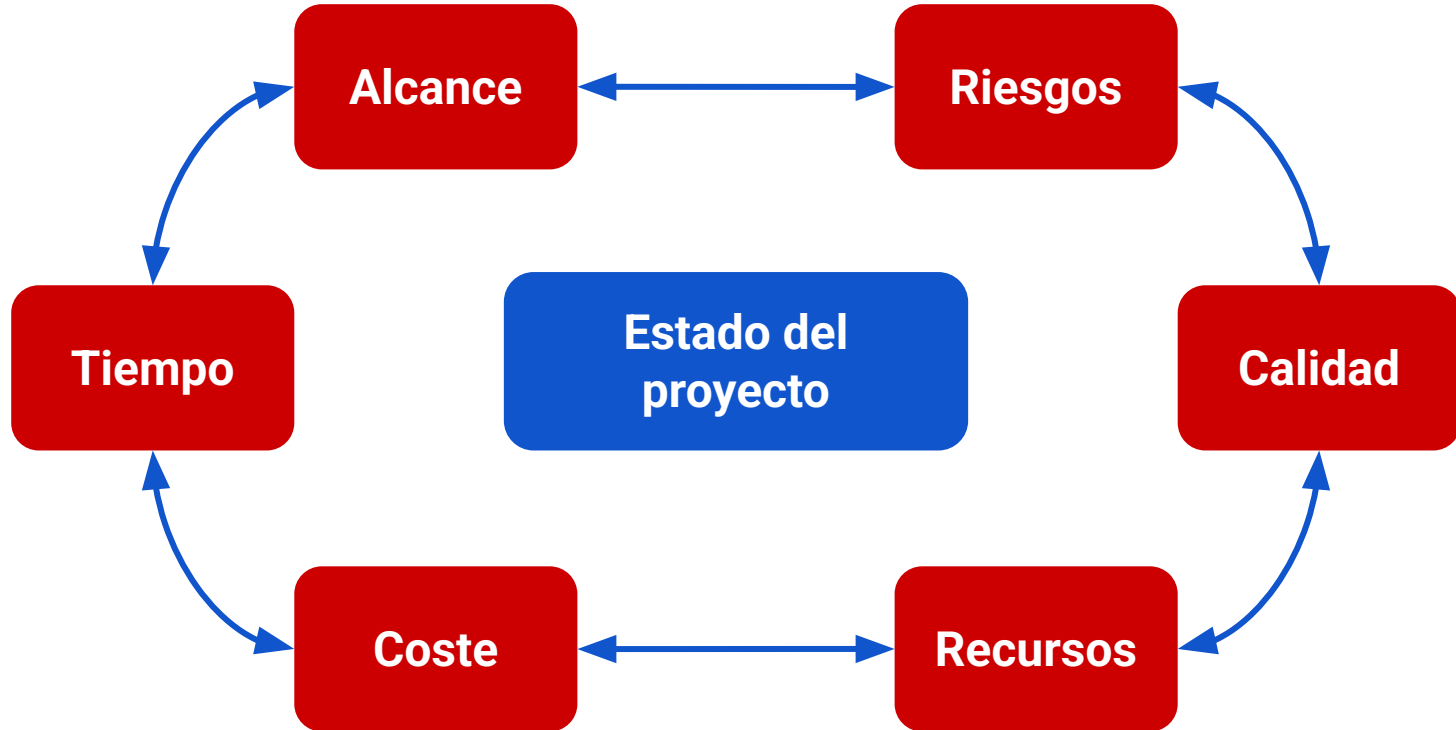
Un proyecto no es una única variable. Es un sistema interdependiente.

Es un sistema donde interactúan:

- Alcance
- Tiempo
- Coste
- Calidad
- Riesgos
- Comunicación

¿Cómo organizamos mentalmente todo esto?

Un proyecto como sistema interdependiente



Project Management Body of Knowledge (PMBOK)

Es un marco profesional de gestión de proyectos

- Guía de referencia internacional para dirección de proyectos.
- Publicada por el Project Management Institute (PMI).
- Actualizada periódicamente (última edición reciente).
- Existe una extensión específica para proyectos de software.

¿Qué es?

- Un cuerpo estructurado de conocimiento.
- Organiza qué debe gestionarse en un proyecto profesional.

¿Qué no es?

- No es una metodología.
- No es una receta.
- No es obligatorio aplicarlo formalmente.

PMBOK como sistema

Ciclo de gestión (muy simplificado)

- Inicio
- Planificación
- Ejecución
- Monitorización y control
- Cierre

No son fases lineales, son grupos de procesos que interactúan continuamente.

PMBOK como sistema

Áreas de conocimiento (relevantes para nosotros)

- Alcance
- Tiempo
- Coste
- Calidad
- Riesgos
- Comunicaciones
- Recursos
- Interesados

Cómo piensa PMBOK: estructura de procesos

PMBOK describe cada proceso como un sistema estructurado:

- Entradas
- Herramientas y técnicas
- Salidas

Esto diferencia la gestión estructurada de la improvisación.

Cada decisión profesional tiene información de partida, método de análisis y resultado documentado.

Monitorización y control en PMBOK

En PMBOK existe un grupo de procesos llamado **monitorización y control**

No es accesorio. No es administrativo. Es estructural.

Simplificado a:

1. Medir.
2. Comparar con el plan.
3. Detectar desviación.
4. Analizar impacto.
5. Decidir ajuste.
6. Actualizar el plan.

Conexión directa con la asignatura

Tema del curso	Área PMBOK
T3 – Alcance	Gestión del alcance
T4 – Estimación	Gestión del tiempo y coste
T5 – Riesgos	Gestión de riesgos
T6 – Seguimiento	Monitorización y control
T8 – Calidad	Gestión de calidad

Proyecto de aplicación web

Situación actual (semana 5 de 8):

- 60% de funcionalidades completadas.
- Hitos temporales cumplidos.
- Sin sobrecoste aparente.
- El equipo ha reducido tiempo de pruebas para cumplir plazo.
- Se han dejado tareas de refactorización para el final.
- Han aumentado pequeñas incidencias internas.

¿Dirías que el proyecto va bien?

- ¿Qué dimensión puede estar degradándose?
- ¿Qué está ocurriendo con la calidad?
- ¿Qué puede pasar en las últimas semanas?
- ¿Es sostenible ese ritmo?

¿Qué es realmente desviación?

¿Qué NO es desviación?

- “Vamos un poco justos.”
- “Hemos ido más lentos esta semana.”
- “Esta tarea ha costado más de lo previsto.”

¿Qué SI es desviación?

- El esfuerzo real supera sistemáticamente el estimado.
- Los hitos empiezan a desplazarse.
- El alcance crece sin decisión explícita.
- La calidad cae para cumplir plazo.

La desviación no es un retraso. Es la pérdida de equilibrio del sistema.

¿Qué es una buena métrica?

Si mañana tuvierais que presentar el estado del proyecto, **¿qué me enseñaríais?**

Una métrica útil:

- **Detecta desviación:** Si el valor cambia, algo está pasando.
- **Permite decidir:** Si el número no cambia ninguna acción, no sirve.
- **Vinculada a una dimensión del sistema:** Debe conectar con alcance, tiempo, coste, calidad, riesgos.

Una métrica es buena si cambia una decisión.

Proyecto de aplicación web

Situación actual (semana 5 de 8):

- 70% tareas completadas.
- 40% funcionalidades integradas.
- 15 incidencias abiertas.
- Hito desplazado 1 semana.

¿Va bien o mal?

- ¿Qué significa 70% tareas completadas si solo 40% está integrado?
- ¿Qué implica 15 incidencias abiertas?
- ¿Un desplazamiento de 1 semana es grave o no?
- ¿Qué dimensión del sistema está afectada?

El peligro de medir mal

No todo lo que se puede medir importa, y no todo lo que importa es fácil de medir.

- ¿Es buena métrica contar líneas de código?
 - Más líneas puede significar: peor diseño, código redundante, más complejidad.
- ¿Más commits implica más progreso?
 - Puede indicar: fragmentación, microcambios, sin relación con valor real.
- ¿Cerrar tareas implica valor entregado?
 - Si no están integradas o probadas no generan valor.

Proyecto de aplicación web

Equipo A:

- 500 commits,
- 100% tareas cerradas,
- sin versión estable.

Equipo B:

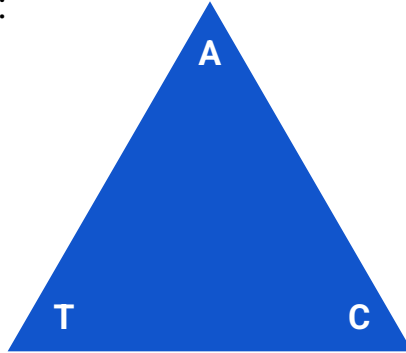
- 1 versión estable,
- menos tareas cerradas,
- menos commits.

¿Quién va mejor?

Earned Value Management (EVM)

Si queremos medir alcance, tiempo y coste simultáneamente, necesitamos algo más que “% completado”.

EVM formaliza con números el triángulo:



EVM convierte el equilibrio del proyecto en magnitudes medibles.

Definiciones formales en EVM

En EVM trabajamos con tres magnitudes básicas.

- **Planned Value (PV):** Valor presupuestado del trabajo que debería haberse completado hasta una fecha determinada.
 - Si el proyecto cuesta 10k€ y en la semana 4 debería estar al 40%: $PV = 10k \times 0.40 = 4k \text{ €}$
- **Earned Value (EV):** Valor presupuestado del trabajo que realmente se ha completado.
 - Si realmente se ha completado el 30% del proyecto: $EV = 10k \times 0.30 = 3k \text{ €}$
- **Actual Cost (AC):** Coste real incurrido hasta la fecha.
 - Si el equipo ha gastado 4k5 € en salarios hasta la semana 4: $AC = 4k5 \text{ €}$

Fórmulas fundamentales en EVM

- **Schedule Variance (SV):** Indica desviación en tiempo (en términos monetarios).
 - **$SV = EV - PV$**
 - Si: $EV = 3k$, $PV = 4k$
 - $SV = 3k - 4k = -1k$
 - Negativo $\rightarrow$ retraso.
- **Cost Variance (CV):** indica desviación en costes.
 - **$CV = EV - AC$**
 - Si: $EV = 3k$, $AC = 4k5$
 - $CV = 3k - 4k5 = -1k5$
 - Negativo $\rightarrow$ sobrecoste.

Índices de rendimiento en EVM

- **Schedule Performance Index (SPI)**
 - **$SPI = EV / PV$**
 - $SPI = 1 \rightarrow$ en plazo.
 - $SPI < 1 \rightarrow$ retraso.
 - $SPI > 1 \rightarrow$ adelantado.
- **Cost Performance Index (CPI)**
 - **$CPI = EV / AC$**
 - $CPI = 1 \rightarrow$ en presupuesto.
 - $CPI < 1 \rightarrow$ sobrecoste.
 - $CPI > 1 \rightarrow$ ahorro.

¿Es necesario usar siempre EVM formalmente?

No es necesario utilizar EVM formalmente siempre.

Pero debéis entender:

- El proyecto puede ir retrasado y con sobrecoste simultáneamente.
- Puede ir en plazo pero con sobrecoste.
- Puede ir con ahorro pero con retraso.

El sistema puede estar desequilibrado en varias dimensiones a la vez.

Control técnico del producto

- Podemos ir en plazo.
- Podemos estar dentro del presupuesto.
- Y aun así estar construyendo un producto frágil.
 - ¿Puede un proyecto “ir bien” y el código ser un desastre?
 - ¿Puede una mala arquitectura no notarse hasta el final?

La calidad no se controla con intuición. Se controla con evidencia técnica.

Cobertura de código

$$\text{Cobertura} = (\text{Líneas ejecutadas por test} / \text{Líneas ejecutable totales}) * 100$$

- Mide cuánto código ha sido ejecutado al menos una vez por tests.
- Es un indicador de esfuerzo en validación automatizada.

Qué SÍ mide

- Grado de instrumentación de tests.
- Nivel de disciplina en pruebas automatizadas.
- Riesgo de código completamente no verificado.

Qué NO mide

- Calidad de diseño.
- Corrección lógica profunda.
- Calidad de los tests.
- Casos límite mal diseñados.

Tasa de defectos

$$\textit{Tasa de defectos} = \textit{Número de defectos} / \textit{Tamaño software}$$

El tamaño puede medirse como KLOC (mil líneas de código), número de módulos, número de historias implementadas.

Si un módulo tiene:

- 3 defectos por cada 1000 líneas
- otro tiene 15 defectos por cada 1000 líneas

¿Dónde está el problema estructural?

Cada defecto implica tiempo de análisis, tiempo de corrección, nuevas pruebas, posible regresión.

Deuda técnica

Sobrecoste por tarea = Tiempo real actual – Tiempo histórico estimado

La deuda técnica se observa cuando:

- Cada nueva funcionalidad cuesta más que la anterior.
- Aparecen efectos colaterales inesperados.
- El equipo evita tocar ciertas partes del código.
- Se retrasan refactorizaciones “para después”.

Si sistemáticamente crece → hay deuda.

Revisión de código

Entramos en calidad del proceso.

¿Qué mide realmente?

- Discusión técnica.
- Transferencia de conocimiento.
- Prevención temprana de defectos.
- Cohesión del equipo.

Métricas posibles

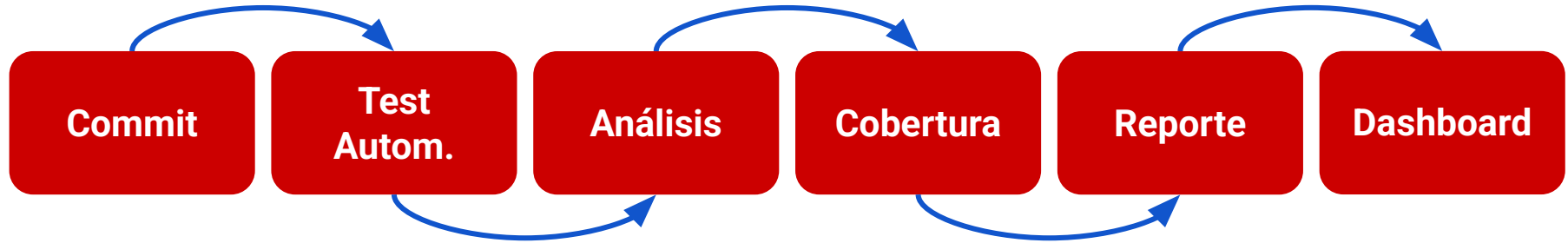
- % Pull Requests revisadas.
- Tiempo medio hasta revisión.
- Número de comentarios técnicos relevantes.
- Tasa de cambios solicitados por revisión.

Métricas de flujo

- **Cycle Time:**
 - Cuánto tarda realmente una tarea.
 - ***Cycle Time=Fecha de finalización-Fecha de inicio***
- **Throughput:**
 - Número de tareas completadas por unidad de tiempo (capacidad real del equipo)
- **WIP (Work In Progress) Limits**
 - Número máximo de tareas simultáneamente abiertas.

Automatización, métricas y profesionalización

- Las métricas no se inventan.
- No se preguntan en reuniones.
- No se estiman “a ojo”.



Las métricas no se inventan. Se extraen automáticamente del sistema de trabajo.

CI / CD

- **CI** → *Continuous Integration* (Integración Continua)
 - Cada cambio se valida automáticamente.
- **CD** → *Continuous Delivery* o *Continuous Deployment*
 - Cada versión es generable automáticamente.

Son prácticas de ingeniería que automatizan la validación del software cada vez que se modifica el código.

Es el mecanismo técnico que automatiza integración, validación, pruebas, generación de métricas.

La ingeniería moderna automatiza el control

Continuous Integration (CI)

- Cada vez que alguien sube código al repositorio,
- El sistema automáticamente:
 - compila el proyecto,
 - ejecuta los tests,
 - analiza calidad,
 - genera cobertura,
 - verifica que todo sigue funcionando.

No se espera al final del sprint. No se espera a “cuando haya tiempo”. Se valida continuamente.

CI convierte cada cambio en un experimento validado automáticamente.

Continuous Delivery / Deployment (CD)

- **Continuous Delivery**

- El sistema genera automáticamente una versión lista para desplegar, pero la publicación puede ser manual.

- **Continuous Deployment**

- Cada cambio validado se despliega automáticamente en producción.

Lo importante es entender:

- Que el producto puede generarse automáticamente.
- Que el estado es siempre verificable.
- Que la integración es continua.

¿Cómo funciona técnicamente CI/CD?

1. Un estudiante hace un commit en GitHub.
2. Se activa un pipeline.
3. El sistema:
 - compila,
 - ejecuta tests,
 - calcula cobertura,
 - ejecuta análisis estático,
 - genera informe.
4. Si algo falla → el equipo lo sabe inmediatamente.

Reuniones y replanificación

Reuniones de seguimiento

Una reunión sana debería:

- revisar métricas reales,
- detectar desviaciones,
- identificar riesgos nuevos,
- decidir ajustes.

Error típico:

- repetir el plan.
- hablar sin datos.
- reuniones donde nadie sabe realmente el estado.

Replanificación

Cuando hay desviación:

- reducir alcance,
- ampliar plazo,
- aumentar recursos.

Se mantiene bloque actual, pero ahora:

- se apoya en dashboards,
- se apoya en métricas reales,
- no en sensaciones.

Conclusión

Riesgo → posibilidad futura.

Seguimiento → detección temprana de que el riesgo se está materializando.

El seguimiento convierte riesgos en información gestionable.

En vuestro proyecto deberíais poder responder a:

- ¿Qué estáis controlando?
- ¿Qué indicadores miráis?
- ¿Qué haríais si detectáis desviación?
- ¿Habéis revisado el plan?